AWS Staging Deployment Guide

This guide details the deployment architecture, configuration steps, and operational procedures for launching the **Intelligent Portfolio Creation Application** in an AWS Staging environment.

---

1. Architecture Overview & VPC Topology



---

2. Secrets & Configuration Management

Never commit production or staging secrets to source control. Use **AWS Systems Manager Parameter Store** (SecureString) or **AWS Secrets Manager**.

Parameter Store Hierarchy (`/portfolio-app/staging/`)

Parameter Name	Type	Description
`/portfolio-app/staging/DATABASE_URL`	SecureString	`postgresql://db_user:db_pass@staging-rds.czx.ap-south-1.rds.amazonaws.com:5432/portfolio_staging`
`/portfolio-app/staging/JWT_SECRET_KEY`	SecureString	64-character cryptographically random secret (`openssl rand -hex 32`)
`/portfolio-app/staging/GEMINI_API_KEY`	SecureString	Google Gemini API Key for LangGraph nodes

Parameter Name	Type	Description
`/portfolio-app/staging/APP_ENV`	String	`staging` (triggers strict fail-closed JWT enforcement)
`/portfolio-app/staging/CORS_ORIGINS`	String	`https://staging.yourdomain.amplifyapp.com,https://stagi`
`/portfolio-app/staging/LOG_LEVEL`	String	`INFO`

EC2 IAM Role Permissions (`PortfolioStagingBackendRole`)

Attach policy granting:

- `ssm:GetParametersByPath` on `arn:aws:ssm:ap-south-1:<account-id>:parameter/portfolio-app/staging/*`
- `logs:CreateLogStream`, `logs:PutLogEvents` on `arn:aws:logs:ap-south-1:<account-id>:log-group:/portfolio-app/staging/*`

---

3. Database Setup & Restore Testing Protocol

Provisioning Amazon RDS PostgreSQL

1. **Engine Version:** PostgreSQL 16.3+
2. **Instance Class:** `db.t4g.micro` (Burstable, ARM-based Graviton2, cost-efficient for staging).
3. **Storage:** 20 GB gp3 with storage autoscaling enabled up to 50 GB.
4. **VPC & Subnet Group:** Place RDS in private subnets across 2 AZs (`ap-south-1a`, `ap-south-1b`).
5. **Security Group:** Inbound TCP 5432 **only** from the EC2 Backend Security Group (`sg-ec2-backend`).
6. **Automated Backups:**
    - Backup retention period: 7 days.
    - Backup window: 03:00–04:00 UTC (off-peak Indian hours).
    - Point-in-time recovery (PITR) enabled.

Database Migration & Seeding

On initial launch or CI/CD deployment:

```
# Run Alembic migrations against staging RDS
DATABASE_URL="postgresql://..." alembic upgrade head

# Seed initial universe & strategies
DATABASE_URL="postgresql://..." python -m backend.seed.seed_data
```

Restore Testing Protocol (AWS Reliability Pillar Compliance)

AWS Well-Architected Framework requires verifying backup recoverability regularly:

1. **Trigger Manual Snapshot:**

    ```
    aws rds create-db-snapshot \
      --db-instance-identifier portfolio-staging-rds \
      --db-snapshot-identifier staging-recovery-drill-$(date +%Y%m%d)
    ```

2. **Restore Snapshot to Temporary Drill Instance:**

```
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier drill-restore-test \
  --db-snapshot-identifier staging-recovery-drill-$(date +%Y%m%d) \
  --db-instance-class db.t4g.micro \
  --publicly-accessible false
```

3. **Run Validation Query against Drill Instance:**

```
psql -h <drill-endpoint> -U db_user -d portfolio_staging -c "SELECT count(*) FROM
securities; SELECT count(*) FROM portfolio_versions;"
```

4. **Teardown Drill Instance:**

```
aws rds delete-db-instance \
  --db-instance-identifier drill-restore-test \
  --skip-final-snapshot
```

---

4. Backend Deployment (FastAPI + LangGraph on EC2)

Provisioning EC2

1. **AMI:** Ubuntu Server 24.04 LTS (HVM), ARM64 or x86_64.
2. **Instance Type:** `t4g.small` (2 vCPU, 2 GB RAM).
3. **Storage:** 30 GB gp3.
4. **Security Group:**
   - Inbound 22: Restricted to Administrator IP.
   - Inbound 443 / 80: Restricted to Application Load Balancer or public if standalone Nginx with Let's Encrypt.

Systemd Service Configuration (`/etc/systemd/system/portfolio-backend.service`)

```
[Unit]
Description=Portfolio Optimization Backend API
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/opt/portfolio-app/backend
EnvironmentFile=/etc/portfolio-backend.env
ExecStart=/opt/portfolio-app/.venv/bin/gunicorn backend.main:app \
    --workers 3 \
    --worker-class uvicorn.workers.UvicornWorker \
    --bind 127.0.0.1:8000 \
    --timeout 120 \
    --access-logfile /var/log/portfolio/access.log \
    --error-logfile /var/log/portfolio/error.log
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Nginx Reverse Proxy Configuration (`/etc/nginx/sites-available/portfolio-api`)

```
server {
    listen 80;
    server_name api-staging.yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api-staging.yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/api-staging.yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api-staging.yourdomain.com/privkey.pem;

    # Timeouts for LangGraph optimization execution
    proxy_connect_timeout 60s;
    proxy_read_timeout 120s;
    proxy_send_timeout 120s;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Health check bypass
    location /health {
        proxy_pass http://127.0.0.1:8000/health;
        access_log off;
    }
}
```

---

5. Frontend Deployment (AWS Amplify Hosting)

1. **Connect GitHub Repository:** Link repository `main` branch to AWS Amplify console.
2. **App Framework:** Select Next.js (SSR / Static).
3. **Build Settings (`amplify.yml`):**

```
version: 1
frontend:
  phases:
    preBuild:
      commands:
        - cd frontend
        - npm ci
    build:
      commands:
        - npm run build
  artifacts:
    baseDirectory: frontend/.next
    files:
      - '**/*'
  cache:
    paths:
      - frontend/node_modules/**/*
      - frontend/.next/cache/**/*
```

4. **Environment Variables in Amplify Console:**
   - `NEXT_PUBLIC_API_URL`: `https://api-staging.yourdomain.com`

---

6. CloudWatch Monitoring & Cost Alerts

CloudWatch Alarms

1. **Backend 5xx Error Rate Alarm:**
    - Metric: Nginx 5xx or FastAPI HTTP 500 responses > 5 over 5 minutes.
    - Action: Send SNS notification to DevOps email.
2. **RDS CPU Utilization Alarm:**
    - Metric: CPUUtilization > 80% for 15 minutes.
3. **EC2 Health Check Failure Alarm:**
    - Metric: StatusCheckFailed_System > 0.

AWS Budgets & Cost Controls

1. Create a monthly cost budget in **AWS Budgets**:
    - Budget Amount: `$25.00 / month`
    - Threshold 1: 80% ($20.00) -> Email alert to Administrator.
    - Threshold 2: 100% ($25.00) -> High-priority notification.
2. Estimated Monthly Staging Cost:
    - EC2 `t4g.small`: ~$12.00/mo
    - RDS PostgreSQL `db.t4g.micro`: ~$13.00/mo
    - Amplify Hosting: ~$2.00/mo
    - **Total Estimated Staging Cost:** ~$27.00/mo (eligible for AWS Free Tier offsets).